Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Kelime gösterimleri (word representation): klasik ve tahmin tabanlı gösterimler.
- Vektör uzay modeli (vector space model), uzaklık ve benzerlik, cosine benzerliği.
- One-hot, doküman tabanlı, bağlam (komşu kelime) tabanlı ve trigram tabanlı gösterimler.
- Seattle-Denver örneği, Pointwise Mutual Information (PMI) ve kelime analojileri.
- Kelime gömmeleri (word embeddings): word2vec (CBOW ve skip-gram), FastText, GloVe, BERT.
- Makine öğrenmesinin NLP açısından hızlı tekrarı: özellik seçimi ve çıkarımı, k-NN, LVQ, çapraz geçerleme, kümeleme.
- Metin sınıflandırma (text categorization): uygulamalar, özellikler, yazar tanıma özellikleri.
- Doküman-terim matrisi, kelime torbası (bag of words), hiyerarşik metin kümeleme, boyutun laneti.
- Abstract Feature Extraction (AFE) yöntemi.
- Kelime koordinatları yöntemi (arama motoru ile birlikte geçme sayıları).
- TF-IDF ağırlıklandırma ve Naive Bayes ile metin sınıflandırma örneği.

Hocanın Özellikle Vurguladığı Kısımlar

- Hangi gösterimin, hangi özelliğin ve hangi yöntemin seçileceği uygulamaya bağlıdır. Hepsi denenmeli, en iyi sonuç vereni seçilmelidir.
- Vektörler normalize edilmeden kullanılırsa uzaklık hesaplarında yanıltıcı sonuçlar çıkar.
- Metinlerde en çok kullanılan benzerlik ölçütü cosine benzerliğidir.
- One-hot gösterimiyle kelimeler arasındaki benzerlik bulunamaz.
- Türkçede embedding'ler kelimelerin kökünden değil, ekli hâlinden çıkarılmalıdır. Anlamsal ilişkiler ekleri de kapsar.
- Özellik sayısı ile eğitim örneği sayısı arasındaki oran çok önemlidir. Hoca bu slaytı "çok önemli" diye işaretledi.
- Test için çapraz geçerleme (cross-validation) kullanılmalıdır.
- Problemi önce keşfetmek gerekir. Örneğin duygu analizinde fiiller önemsiz olabilir, onları özellik olarak almanın anlamı yoktur.
- Doğru özellikler seçilir, oran ayarlanır, gerekirse uzay değiştirilir ve çapraz geçerleme yapılırsa kötü sonuç almak zordur. Yine de veri setinin içeriği belirleyicidir.
- Bir yöntemi "pat diye" seçip kullanmamak gerekir.

Kısa Tekrar Notları

- Klasik gösterimler: count vektörü, TF-IDF. Tahmin tabanlı gösterimler: word2vec, GloVe, BERT, ELMo, FastText ve türevleri.
- Cosine benzerliği: $\cos\theta = \frac{\vec{a}\cdot\vec{b}}{\|\vec{a}\|\|\vec{b}\|}$.
- PMI: $pmi(x;y) = \log\frac{p(x,y)}{p(x)p(y)}$.
- Analoji: $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$.
- Word embedding: düşük boyutlu (genelde 150 ya da 300) ve yoğun (dense) vektör. 2013'te Google'da Mikolov ve ekibi önerdi.

- Skip-gram: girdi merkez kelime, çıktılar çevresindeki kelimeler. CBOW: tam tersi.
- FastText: kelime yerine karakter n-gram'ları kullanır. Yazım hatalı ve sözlükte olmayan kelimelere de vektör üretir.
- GloVe: word2vec yaklaşımına birlikte geçme istatistiklerini ekler.
- BERT: çift yönlü (bidirectional) transformer. Masked Language Modeling (MLM) ve Next Sentence Prediction (NSP) kullanır.
- Özellik seçimi: filtre (filter) ve sarmalayıcı (wrapper) yöntemler. Özellik çıkarımı: PCA, LDA, LSI (uzayı değiştirir).
- AFE: özellik sayısını sınıf sayısına indirir. Yalnızca terim ağırlıklarını ve sınıflar üzerindeki dağılımlarını kullanır.
- TF-IDF: $w = tf \times \log \frac{N}{n}$. Her dokümanda geçen kelimenin ağırlığı sıfır olur.
- Naive Bayes: $v_{NB} = \arg\max_{v_j \in V} P(v_j) \prod_i P(w_i|v_j)$.

Detaylı Açıklamalar

Ders başı duyuruları

- Hoca bir gün önce 97 kişilik bir lisans sınıfının 5 soruluk ödevlerini okumuş. Bilgisayardan okumanın çok zor olduğunu söyledi.
- Bir öğrenci ödevine dönüş alamadığını sordu. Hoca ödevi işlediğini, bilgi vereceğini söyledi.
- Seminer takvimi ilan edildi. İki hafta yerine üç haftaya yayıldı. Sebep: bir haftaya çok fazla sunum yığılıyordu.
- Arada zaman kalırsa eksik konular anlatılacak.
- Seminerlerden önce genel bir vize yapılacak. Amaç ayrıntı sormak değil, genel kavramlardan nelerin akılda kaldığını görmek.
- Makine öğrenmesi dersini almayan öğrencilerden sohbete yazmaları istendi. Kimse yazmadı. Hoca herkesin aldığını kabul edip makine öğrenmesi slaytlarını hızlı geçti.

Kelime gösterimi (word representation)

- Bir dokümanla işlem yapmak için ondan özellikler çıkarılmalı ve bu özellikler vektör hâline getirilmelidir.
- Hangi özelliklerin alınacağı uygulamaya göre değişir. Gösterim biçimi ise aynıdır.
- Kelimeler anlamlarını dikkate alan bir gösterimle ya da yalnızca sayısal varlıklarıyla gösterilebilir.
- Kelimelerin işlenebilir formattaki hâline kelime gösterimi denir.
- İki ana grup:
  - Klasik, frekans tabanlı gösterimler: count vektörü, TF-IDF (Term Frequency - Inverse Document Frequency).
  - Tahmin tabanlı gösterimler: derin öğrenmeyle hayatımıza girdi. word2vec, GloVe, BERT, ELMo, BERT'ten türetilen ALBERT ve RoBERTa, FastText.
- Eskiden yapay sinir ağlarında bir girdi, bir gizli ve bir çıktı katmanı vardı. Gizli katmanların derinliği artınca derin öğrenme ortaya çıktı.
- Arka planda karmaşık işler yapılsa da sonuçta elde edilen şey basit bir vektördür. Bu vektör bir kelimeyi, bir cümleyi ya da bir dokümanı anlatır.

Vektör uzay modeli, uzaklık ve benzerlik

- Vektör uzay modelinde her kelime, cümle, paragraf ya da doküman bir vektördür. 100 elemanlı bir vektör, 100 boyutlu uzayda bir noktadır.
- İki vektörün benzerliğini bulmak, aralarındaki açıyı bulmak demektir. Açı hesabı maliyetli olduğu için çeşitli metrikler geliştirilmiştir.
- Uzaklık (distance) ve benzerlik (similarity) ters orantılıdır. İkisi de $[0, 1]$ aralığına alınırsa:
  - Uzaklık 0 ise benzerlik 1'dir. İki vektör aynıdır.
  - Benzerlik 0 ise uzaklık 1'dir.
- Count ve TF örneği:
  - Türkçede 29 harf vardır. Bir cümle için 29 boyutlu bir vektör oluşturulur. Önce büyük-küçük harf ayrımının önemli olup olmadığına karar verilir. Önemli değilse hepsi kü-

çük harfe çevrilir.
  - Count: her harfin cümlede kaç kez geçtiği (örneğin “a” 5 kez).
  - TF (terim frekansı): her sayım en sık geçen terimin sayımına bölünür. En sık terim “e” 10 kez geçtiyse, 5 kez geçen “a” için $5/10$ yazılır.
  - Böylece değerler 0 ile 1 arasına sıkışır.
- Normalizasyonun önemi: boşluk karakteri de özellik olsun. Bir dokümanda 10.250 boşluk, diğer karakterlerden 3-7 tane olabilir. Uzaklık hesabında kareler alınır. 5’in karesiyle 10 binin karesi arasındaki fark yanıltıcı sonuç verir.

Kelime vektörleriyle analoji

- Erkek ve kadın vektörleri ile kral vektörü varsa kraliçe vektörü tahmin edilebilir.
- Erkek ile kadın arasındaki fark, kral ile kraliçe arasındaki farka eşittir. Cebirsel işlem yapıyormuş gibi düşünülür.
- Metinlerde en çok cosine benzerliği kullanılır. Akademik çalışmalarda da hep cosine benzerliği görülür.
- word2vec, GloVe, BERT gibi derin öğrenme tabanlı kelime vektörleri eğiticisiz (unsupervised) öğrenilir. Sınıf bilgisine gerek yoktur.
- Her kelime için belirli sayıda özellik öğrenilir. Özellik sayısına kullanıcı karar verir. Her özellik anlamsal bir bilgi taşır.

One-hot gösterimi

- En basit gösterimdir. Derin öğrenme modellerinin girdi katmanına da vektörün one-hot hâli verilir.
- Sözlük boyutu $V = 10.000$ ise her kelime 10.000 boyutlu bir vektördür.
  - Sözlükteki 1. kelime “a” ise vektörün 1. gözü 1, diğerleri 0 olur.
  - 2. kelime “adam” ise 2. gözü 1 olur.
  - 10.000. kelime “zil” ise son gözü 1 olur.
- Seyrek (sparse) bir gösterimdir.
- One-hot ile kelimeler arasındaki benzerlik bulunamaz. Hiçbir kelime diğerine benzemez.

Diğer gösterim biçimleri

- Doküman tabanlı gösterim: kelime, geçtiği dokümanlara göre gösterilir.
  - Örnek: 12 doküman var. “aba” kelimesi 1. dokümanda hiç, 2. dokümanda 1, 4. dokümanda 1, 7. dokümanda 2, 9. dokümanda 1, 11. dokümanda 1 kez geçiyor.
  - Gözlere count ya da TF değerleri yazılabilir.
- Bağlam (komşu kelime) tabanlı gösterim:
  - Cümle: “Yellow banana grows on trees in Africa”. “banana” için vektör oluşturuluyor.
  - Kelimenin solundaki 5 ve sağındaki 5 kelimeye bakılır. Her göz bir (kelime, konum) çiftidir: (yellow, -1), (grows, +1), (on, +2), (trees, +3), (in, +4), (Africa, +5).
  - Dokümanın başka bir yerinde de “banana grows” geçiyorsa (grows, +1) gözü 2 olur. Yani banana en çok “grows” ile birlikte kullanılmıştır.
  - Başka bir kelime (örneğin “an orange”) için (an, -1) gözü 1, banana için 0 olur.
- Trigram tabanlı gösterim:
  - Vektörün gözleri, Türkçede en sık kullanılan ilk 100 ya da 500 trigram olabilir.
  - “banana” kelimesinin trigramları: “ban”, “ana”, “nan”, “ana”, “na_”. “ana” gözü 2 olur.
  - Bigram gibi başka seçenekler de kullanılabilir.
- İlgililik (relatedness): one-hot ile bulunamayan benzerlik, bağlam ve trigram vektörleriyle bulunabilir. Aynı kelimelerle kullanılan kelimeler birbirine yakındır. Örneğin “Afrika”, “ağaç”, “yetişmek” kelimeleriyle geçen iki meyve aynı konseptte yer alır.

Cosine benzerliği

$$\cos\theta = \frac{\vec{a}\cdot\vec{b}}{\|\vec{a}\|\|\vec{b}\|}$$

- Pay: iki vektörün skaler çarpımı, yani karşılıklı elemanların çarpımlarının toplamı.

- Payda: vektör boylarının çarpımı. Vektörün boyu, elemanlarının karelerinin toplamının kareköküdür.
- Kral-kraliçe benzerliği ile erkek-kadın benzerliği birbirine yakın çıkar. Buradan kralın erkek, kraliçenin kadın olduğu çıkarılabilir.

Seattle-Denver örneği

- Dört doküman:
  - Doküman 1: “seattle seahawks jerseys”
  - Doküman 2: “seattle seahawks highlights”
  - Doküman 3: “denver broncos jerseys”
  - Doküman 4: “denver broncos highlights”
- Soru: Seattle ile Denver, Seahawks ile Broncos benzer kelimeler midir?
- Konu benzerliği (topic similarity): kelimenin dokümanda geçip geçmediği kullanılır. Vektörler 4 boyutludur.

Kelime	D1	D2	D3	D4
seattle	1	1	0	0
seahawks	1	1	0	0
denver	0	0	1	1
broncos	0	0	1	1

- seattle ile seahawks arasındaki cosine benzerliği: $\frac{1\cdot1+1\cdot1+0+0}{\sqrt{2}\sqrt{2}} = 1$. İki kelime birbiriyle örtüşür. denver ve broncos için de aynı sonuç çıkar.
- Bağlam vektörleri (typical, by-type): kelimenin 2 solu ve 2 sağı kullanılır.

Kelime	(seattle,-1)	(seahawks,-1)	(denver,-1)	(broncos,-1)	(jerseys,+1)	(jerseys,+2)	(highlights,+1)	(highlights,+2)
seattle	0	2	0	0	0	1	0	1
seahawks	2	0	0	0	1	0	1	0
denver	0	0	0	2	0	1	0	1
broncos	0	0	2	0	1	0	1	0

- Bu gösterimde seattle denver’a, seahawks broncos’a benzer çıkar. İkisi de şehirdir, ikisi de takımdır.
- Sonuç: vektörlerin nasıl oluşturulduğu bulunan benzerliği değiştirir. Seçim uygulamanın amacına göre yapılır.
- Trigram ile “seattle” ve “settle” kelimelerinin benzerliği de ölçülebilir.

PMI ve örnek sonuçlar

- Sık kullanılan bir başka ölçüt Pointwise Mutual Information (PMI):

$$pmi(x;y) = \log \frac{p(x,y)}{p(x)p(y)}$$

- İki kelimenin birlikte geçme olasılığı, ayrı ayrı olasılıklarının çarpımına bölünür ve logaritması alınır.
- Slayttaki uygulama: kelime-bağlam çiftleri PMI ile ölçülüp en benzer kelimeler listelenmiş.

Sorgu	En benzer kelimeler (benzerlik katsayısı)
sydney	melbourne 0,44, brisbane 0,41, perth 0,34, adelaide 0,29, auckland 0,25

Sorgu	En benzer kelimeler (benzerlik katsayısı)
batman	spiderman 0,14, superman 0,14, ghostbusters 0,10, tinkerbell 0,09, starwars 0,08
java	c 0,16, javascript 0,15, powershell 0,11, python 0,10, vb 0,09
pasta	spaghetti 0,18, lasagna 0,15, macaroni 0,11, salad 0,10, casserole 0,10

- Değerler 1'e yakın değil ama en yakın çıkan kelimeler hep aynı alandan: Avustralya şehirleri, çizgi roman ve film kahramanları, programlama dilleri, yemekler. "Sydney" için "book", "pencil" gibi ilgisiz kelimeler çıkmıyor.
- Sonuçlar kullanılan derleme göre değişir.

Kelime analojileri

- King ile Queen arasındaki ilişki, Man ile Woman arasındaki ilişkiyle aynıdır.
- Fiillerde de vardır: walking-walked ile swimming-swam.
- Ülke-başkent: Spain-Madrid ilişkisi verilince Italy için Rome, China için Beijing bulunur.
- $\vec{king} - \vec{man} + \vec{woman}$ işlemi tam olarak $\vec{queen}$ vektörünü vermez. Ona çok yakın bir vektör verir. Bu yüzden "yaklaşık" denir.

Kelime gömmeleri (word embeddings)

- Önceki gösterimlerin boyutları çok yüksekti (sözlük boyutu kadar) ve çok seyrekti.
- Aynı sezgileri kullanan ama daha düşük boyutlu ve daha yoğun vektörler geliştirildi.
  - Boyut kullanıcı tarafından seçilir: 100, 150, 300 vb. Çalışmalarda genelde 150 ve 300 tercih edilir.
  - Boyut çok küçükse başarı düşer. Çok büyükse anlamsal ilişkiler kaybolabilir.
  - Yoğun: her gözde sıfırdan farklı bir değer vardır ve elemanlar arasındaki ilişkiler güçlüdür.
- Word embedding kavramını 2013'te Google'da çalışan Mikolov ve ekibi ortaya attı.
- Arka planda bir matris ayrıştırma (matrix factorization) yapılır. Matris anlamlı parçalara bölünür. GloVe ve LDA'da da kullanılan mantık budur.
- Arkasında bir yapay sinir ağı vardır: girdi katmanı, gizli katman(lar) ve çıktı katmanı. Hem encoder hem decoder gibi çalışır.
- Hoca model içine girmedi. Python'da modeller hazırdır, katmanlar eklenip çalıştırılır.
- Kullanım örneği, bilgi erişimi (information retrieval):
  - Geleneksel sistemler terim eşleştirme (term matching) yapar. Sorgu kelimesi geçen her pasajı getirir. Kelime yalnızca bir ad olarak geçse bile.
  - Embedding kullanılırsa dokümanların içerikçe yakınlığı da kontrol edilir. İlgisiz pasaj getirilmez.
- Embedding matrisi: M kelime ve D boyutlu vektörler için D satır, M sütunluk bir matristir. Her sütun bir kelimenin vektörüdür. Bir cümle de bu vektörlerle matris hâline getirilebilir.

word2vec

- Kelimeyi vektörle ifade eder. İki modeli vardır: CBOW (Continuous Bag of Words) ve skip-gram. İkisi aynı yapıdadır ama ters yönde çalışır.
- Kavramlar:
  - Merkez kelime: embedding'i oluşturulacak kelime.
  - Çerçeve (pencere, window) boyutu: merkez kelimenin sağından ve solundan kaç kelime alınacağı. Sağdan 3, soldan 3 kelime alınırsa pencere boyutu 7 olur.
  - Çevreleyen kelimeler: merkez kelimenin etrafındaki kelimeler.
- Türkçe için önemli nokta: "korkma" kelimesinin kökü "kork"tur. Ama embedding'ler kök yerine ekli hâlleriyle çıkarılmalıdır, çünkü anlamsal ilişkiler ekleri de kapsar. Bu yüzden Türkçede embedding sayısı çok fazladır.

- Yapı: bir girdi katmanı, bir gizli katman ve bir çıktı katmanından oluşan yapay sinir ağı.
  - Hiperparametreler: pencere genişliği, embedding boyutu vb.
  - Embedding boyutu, gizli katmandaki nöron sayısına eşittir. Bütün kelimelerin vektör boyutu aynıdır.
  - Örnek: girdi one-hot (10.000 boyut, "ants" kelimesinin gözü 1). Gizli katman 300 nöron. Çıktıda softmax sınıflandırıcı, 10.000 nöron (her biri sözlükteki bir kelime).
- Kullanım alanları: kelime ve cümle benzerliği, doküman özetleme vb.

Skip-gram ve CBOW

- Skip-gram:
  - Girdi merkez kelimedir, çıktılar çevresindeki kelimelerdir.
  - Girdi ve çıktılar olasılıksal olarak birbirine benzetilerek kelime anlamsal olarak en uygun biçimde temsil edilir.
  - Örnek: "Korkma sönmez bu şafaklarda yüzen al sancak". Girdi "şafaklarda". Pencere boyutu 7 ise çıktılar "korkma", "sönmez", "bu", "yüzen", "al", "sancak" olur.
  - Girdi, "şafaklarda" kelimesinin one-hot vektörüdür. Çıktılar çevreleyen kelimelerin one-hot vektörleridir.
  - Derlemde "şafaklarda" her geçtiğinde, sağındaki ve solundaki kelimelere göre vektörü güncellenir. Derlem bitince her kelime için bir embedding çıkar.
- CBOW:
  - Tam tersidir. Girdiler çevreleyen kelimelerin one-hot vektörleri, çıktı merkez kelimenin one-hot vektörüdür.
- Hocanın verdiği karşılaştırma:

Özellik	CBOW	Skip-gram
Veri seti	Küçük veri setlerinde daha iyi	Büyük veri setlerinde daha iyi
İşlem gücü	Az	Daha fazla
Çok anlamlı kelimeler ("yüz")	Başarılı değil	Daha iyi embedding

FastText

- 2016'da Facebook tarafından word2vec'in uzantısı olarak geliştirildi.
- Yapay sinir ağına kelimeler değil, karakter n-gram'ları verilir.
- Örnek: "dogal" (yumuşak g'siz yazılmış "doğal") için trigramlar "dog", "oga", "gal". Kelimenin vektörü, trigram vektörlerinin toplamı ya da ortalaması alınarak veya arka arkaya eklenerek oluşturulur.
- Neden tercih edilir?
  - Yazım hatalı kelime: "dogal" diye bir Türkçe kelime yoktur. Diğer modeller buna vektör veremez. Ya sıfır ya da rastgele vektör atanır. FastText ise trigram vektörlerinden gerçek kelimeye çok uzak olmayan bir vektör üretir.
  - Derlemde hiç geçmeyen kelimeler: trigramlarına ayrılarak vektör atanabilir.
  - Az geçen kelimeler word2vec'e göre daha iyi ifade edilir.
- Dezavantaj: n-gram sayısı kelime sayısından çok fazladır. Eğitim uzun sürer.

GloVe

- Pennington ve arkadaşları tarafından önerildi.
- Skip-gram ve CBOW anlamsal bilgiyi yakalar ama birlikte kullanılma istatistiklerini kullanmaz.
- Matris ayrıştırma yöntemleri istatistik bilgisini kullanır ama anlamsal ilişkileri yakalayamaz.
- GloVe olasılık istatistiklerinden yararlanarak yeni bir amaç (objective) fonksiyonu oluşturur:

$$J(\theta) = \frac{1}{2} \sum_{i,j=1}^{W} f(P_{ij})(u_i^T v_j - \log P_{ij})^2$$

- P_{ij}: iki kelimenin birlikte geçme olasılığı. u_i ve v_j: kelime vektörleri. Transpoz, iki vektörü çarpabilmek için alınır.
- Birlikte bulunma olasılığı yüksek olan kelimeler öğrenmede daha önemli rol oynar.
- Hocaya göre olasılıkları hesaplayan modelin kendisidir. Formülün ayrıntısı kullanıcı için kritik değildir.
- Örnek: London-England ilişkisi verilip Iran için başkent soruluyor. Sonuçlar: Tahran, Bağdat, Irak, Kahire, Erbil. En yüksek Tahran. Bağdat'ın yakın çıkması, bu kelimelerin derlemde çok birlikte geçmesinden kaynaklanır.

Hazır eğitilmiş kelime vektörleri

- Kelime vektörlerini öğrenmek maliyetlidir. Çok GPU ve çok büyük derlem gerekir.
- Bu yüzden birçok dil (Türkçe dahil) için hazır vektörler paylaşılmıştır.

Kaynak	Token sayısı	Tekil kelime sayısı	Vektör boyutu
Wikipedia 2014 + Gigaword 5	6 milyar	400 bin	50, 100, 200, 300
Common Crawl 1	42 milyar	1,9 milyon	300
Common Crawl 2	840 milyar	2,2 milyon	300
Twitter (2 milyar tweet)	27 milyar	1,2 milyon	25, 50, 100, 200
Google News	100 milyar	3 milyon	300
Facebook (294 dil)			300

- Türkçe için özel eğitilmiş kelime vektörleri de vardır.
- Loodos adlı bir firma tamamen Türkçe üzerine çalışıyor. BERT ve türevleri için Türkçe modelleri açık paylaşıyor. Zemberek'in bazı modüllerini Python'a geçirip açık kaynak olarak paylaşıyor.

BERT

- 2018'de Google tarafından yayımlandı. Transformer'larla birlikte çok popüler oldu.
- Açılımı: Bidirectional Encoder Representations from Transformers.
- Hocaya göre ürettiği vektörler GloVe ve word2vec'ten daha iyidir. Daha fazla anlamsal ilişki çıkarır.
- Cümleyi hem soldan sağa hem sağdan sola değerlendirir.
  - RNN'de cümlenin sonuna gelindiğinde baştaki kelimeyle ilişki unutulabilir.
  - BERT'te cümlenin anlamı ve kelimeler arası ilişkiler daha iyi çıkarılır.
- Eğitim: Wikipedia (2,5 milyar kelime) ile bir kitap derlemi birleştirilmiş. BERT-Large ve BERT-Base iki model. Biri 16 TPU, diğeri 4 TPU üzerinde 4 gün eğitilmiş.
- Kullanım: önceden eğitilmiş (pretrained) model alınır. Probleme göre üstüne ek katmanlar eklenir ve ince ayar (fine-tuning) yapılır. Chatbot ve metin sınıflandırmada kullanılır.
- İki strateji:
  - Masked Language Modeling (MLM): cümledeki kelimelerin %15'i seçilir.
    - Bunların %80'i [MASK] etiketiyle kapatılır.
    - %10'u rastgele başka bir kelimeyle değiştirilir.
    - %10'u değiştirilmeden bırakılır.
    - Amaç, maskelenen yere hangi kelimelerin gelebileceğini tahmin etmeyi öğrenmektir.
  - Next Sentence Prediction (NSP): eğitimde ikili cümleler verilir. İkinci cümlenin birincinin devamı olup olmadığı tahmin edilir. Böylece cümleler arası ilişki de öğrenilir.
- NSP sayesinde soru cevaplama (question answering) sistemlerinde başarılıdır.

Makine öğrenmesinin hızlı tekrarı

- Eğiticili (supervised) yöntemlerde etiket bilgisi vardır. Eğiticisiz (unsupervised) yöntemlerde yoktur.
- Dokümanlardan özellik vektörü çıkarılır. Özellik adayları:
  - Kelimeler ve frekansları, anlamsal ilişkiler.
  - Dil modeliyle ilgili bilgiler, n-gram'lar.

  - POS etiketleri (isim, fiil, sıfat).
  - Gramer kuralları: kurallı cümle, devrik cümle, soru cümlesi.
- Özellik çok fazlaysa gereksizler de girer. İki seçenek vardır:
  - Özellik seçimi (feature selection): en ayırt edici özellikleri seçmek.
  - Özellik çıkarımı (feature extraction): çalışılan uzayı değiştirmek. Verinin taşıdığı bilgi değişmez, sayısal değerleri değişir. Böylece veriler doğrusal bir eğriyle ya da bir kernel ile ayrılabilir.
- Uygulama örnekleri:
  - Mailin spam olup olmaması.
  - Çin'de bir pilot çalışma: son 20-30 yıldaki hafif suçlarda (hırsızlık gibi, cinayet değil) hangi delillerle hangi cezaların verildiği öğretilmiş. Aynı şartlarla gelen bir davaya sistem karar öneriyor. Yalnızca pilot olarak, aynı cevapların verilip verilmediği denenmiş.
- Hocanın "çok önemli" dediği slayt:
  - x ekseni özellik sayısı (10'dan 10^5'e kadar), y ekseni eğitim örneği sayısı.
  - Farklı alanlar (sistem teşhisi, OCR, el yazısı tanıma, metin kategorizasyonu, pazar analizi, biyoinformatik, makine görmesi) farklı aralıklarda yer alır.
  - Örneğin biyoinformatikte girdi bir DNA dizilimi olduğu için özellik sayısı çok fazladır ama daha az örnekle eğitilebilir.
  - Teoride verilen bu sayılara pratikte ulaşılamayabilir. Ama aradaki oranı yakalamak çok önemlidir.

Özellik seçimi ve çıkarımı

- Filtre (filter) yöntemleri: her özellikle sistem ayrı ayrı çalıştırılır. Başarıya göre sıralanır, en iyileri seçilir.
  - En çok kullanılanı bilgi kazancıdır (information gain). Entropiye dayanır. Weka'da da bulunur.
- Sarmalayıcı (wrapper) yöntemler: özellik alt kümeleri oluşturulur.
  - 5 özellik varsa 1'li, 2'li, ..., 5'li bütün kombinasyonlar denenir. En yüksek başarıyı veren alt küme seçilir.
  - Bütün özelliklerden başlayıp azaltarak ya da tek özellikten başlayıp en başarılıları birleştirerek ilerlenebilir.
- Yeni özellik çıkarımı: iki özelliği birleştirerek yeni özellik yapmak. PCA (Principal Component Analysis) ve LDA (Linear Discriminant Analysis).
  - Hocaya göre PCA metinlerde pek iyi sonuç vermez. LDA iyi sonuç verir ama etiketli bir yöntemdir.

Sınıflandırma yöntemleri

- Hangi yöntemin seçileceğine baştan karar verilemez. Olasılık tabanlı, ağaç tabanlı ve kernel tabanlı yöntemlerden birer model kurulur. Veri hangisine uygunsa onunla devam edilir.
- k en yakın komşu (k-NN):
  - Yeni gelen örneğin bütün verilere uzaklığı ölçülür ve kısadan uzuna sıralanır.
  - k kaç seçildiyse o kadar komşuya bakılır. $k = 1$ ise en yakın komşunun sınıfı verilir.
  - $k = 3$ ve komşuların 2'si B, 1'i A ise örnek B sınıfına atanır.
  - k değerinin seçimi önemli bir parametredir.
- Learning Vector Quantization (LVQ):
  - Etiketli verilerle çalışır. Prototip vektörler rastgele başlatılır.
  - Bir örneğe en yakın prototip bulunur. Sınıfları aynıysa prototip ödüllendirilir ve örneğe yaklaştırılır. Farklıysa cezalandırılır ve uzaklaştırılır.
- Çapraz geçerleme (cross-validation):
  - Eski çalışmalar veriyi %70-%30 ya da %80-%20 bölüp test ederdi.
  - Sorun: eğitim kümesine kolay örnekler düşerse sistem ezberler ve testte başarısız olur. Zor örneklerin hepsi teste düşerse iyi model de kötü görünür.
  - k-katlı çapraz geçerleme (k-fold): veri k parçaya bölünür. Her seferinde bir parça test, kalanlar eğitim için kullanılır. k başarının ortalaması sistemin başarısıdır.
- Her veri seti farklı bir yöntemde daha iyi sonuç verir. Özellik sayısı, sınıf sayısı ve örnek sayısıyla kesin bir ilişki kurulamaz. Verinin içeriği de belirleyicidir. Bu yüzden uzaklık,

ağaç, kernel ve istatistik tabanlı yöntemler denenmelidir.

Kümeleme

- k-means (k ortalamalar):
  - Küme sayısı k kullanıcı tarafından belirlenir. Etiketler bilinmese de tahmini bir sayı verilir (örneğin gazete için ekonomi, siyaset, teknik, magazin, sağlık gibi 10 civarı küme).
  - Adımlar:
    1. k küme merkezi (centroid) rastgele ya da verilerden seçilerek atanır.
    2. Her nokta en yakın merkezin kümesine atanır.
    3. Her kümenin yeni merkezi, kümedeki noktaların ortalaması olarak hesaplanır.
    4. 2. ve 3. adımlar tekrarlanır.
  - Durdurma: belirli bir adım sayısı, kümeler arasında yer değişikliği olmaması ya da küme içi varyansın en küçük, kümeler arası varyansın en büyük olması.
  - Kümelerin adını insan verir. Örneğin insülin değerleri yüksek hastalar için doktor "şeker hastalığı" der. NLP'de siyasetle ilgili kelimeler içeren kümeye "siyaset", sporla ilgili olana "spor" etiketi verilir.
- X-means: k değerini algoritma kendisi belirler.
- DBSCAN: doğrusal olarak ayrılamayan, iç içe geçmiş verilerde iyi sonuç verir.
- Self-organizing map (SOM): k-means'e benzer. k-means bir verinin bütün kümelere uzaklığını ölçerken SOM yalnızca belirli bir komşuluk mesafesindeki birimlere bakar. Tek ve iki boyutlu türleri vardır.
- Yarı eğiticili (semi-supervised) yöntemler: çok az etiket varsa etiketli yöntem kullanılamaz. Ama bu değerli etiketler de kaybedilmek istenmez. En popüler olanı YATSI algoritmasıdır.

Metin sınıflandırma (text categorization)

- Elde bir metin var ve sınıflandırılıyor. Uygulama örnekleri:
  - Haber metinlerini başlığa ya da içeriğe göre ayırmak.
  - Tweet'leri konuya ya da duyguya göre ayırmak.
  - Bir tweet'in nefret söylemi olup olmadığını bulmak.
  - Metnin yazarını, yazarın cinsiyetini, dokümanın dilini bulmak.
  - Bir kodun hangi programlama dilinde yazıldığını, içinde başka dillerde kod olup olmadığını bulmak.
  - Bir kodun exe kodu mu, normal kod mu olduğunu anlamak.
  - Mailin spam olup olmadığını bulmak.
- Eğiticili, eğiticisiz ve yarı eğiticili yöntemlerden biri kullanılır.
- Özellikler problemi çözmeye yönelik seçilmelidir. Her özellik her uygulamada iyi sonuç vermez.
- Özellik adayları: kelimeler, POS etiketleri, n-gram'lar, ekler ve türleri, noktalama işaretleri, yalnızca işlevsel kelimeler (function words), doğrudan anlatım ifadeleri ("söyledi", "dedi", "ona göre").

Yazar tanıma özellikleri

- Dokümandaki cümle sayısı: bazı yazarlar uzun, bazıları kısa cümle kurar. Bir A4 sayfada 15 de 50 de cümle olabilir. Bu yazarın stilidir.
- Kullanılan toplam kelime sayısı (token). Çok çekim eki kullanan yazarlarda ("uygarlaştıramadıklarımızdan mısınız" gibi) kelime sayısı azdır.
- Cümledeki ortalama kelime sayısı, kelime uzunluğu, farklı kelime sayısı.
- Kelime zenginliği (word richness): aynı kavram için farklı kelimeler kullanmak. Örneğin bir paragrafta "okul", diğerinde "mektep".
- Cümledeki isim, fiil, sıfat, zarf, edat, zamir, bağlaç sayıları.
- Noktalama işaretlerinin sayıları: nokta, virgül, iki nokta, soru işareti, ünlem.
- Kurallı ve devrik cümle sayıları.
- Üç nokta kullanımı: bazı yazarlar cümleyi bitirmez, okura bırakır.

Metin gösterimi ve doküman-terim matrisi

- Metin vektör hâline getirilir. Her özellik bir kelimeye karşılık gelir. Gözlere count, TF, olasılık ya da ağırlıklandırılmış değerler yazılabilir.
- Stop words, sayılar, nokta, tire, virgül gibi anlamsız öğeler atılabilir. Bunlar yalnızca vektörün boyutunu uzatır.
- Doküman-terim matrisi: satırlar dokümanlar, sütunlar terimler olabilir (ya da tersi). d_{22}: 2. dokümanda 2. terimin kaç kez geçtiği.
- Bütün terimler bütün dokümanlarda geçmediği için seyrektir. Gerekirse yumuşatma teknikleri uygulanır.
- N-gram özellikleri: eskiden token ayırma, kök bulma, n-gram çıkarma için kod yazılırdı. Bugün Python kütüphaneleriyle tek satırda yapılıyor.
- Örnek: iki dokümandan kelime bigram'ları çıkarılmış. "army troops" 1. dokümanda bir kez geçmiş, 2. dokümanda hiç yok.
- Özellik sayısını azaltmak için stop words silinir ve her şey küçük harfe çevrilir. Aksi hâlde "Army" ve "army" ayrı özellik olur.

Hiyerarşik metin kümeleme

- Etiketsiz 9 doküman vektöre dönüştürülür. Başta her doküman ayrı bir kümedir.
- Benzer dokümanlar birleştirilir: önce D3 ile D7, sonra D1 ile D2 vb.
- İstenen küme sayısında (4, 3, 2) kesilir.
- Faydası: bazı dokümanlar birden fazla konu içerir. Örneğin D3 ve D7 siyaset, D4 hem siyaset hem ekonomi, D9 ekonomi. Siyaset ve ekonomi iç içe olduğu için düz ayırma başarısız olabilir. Hiyerarşik yapıda farklı seviyeler incelenebilir.
- Durdurma: bir eşik değeri verilir. Kümeler arası benzerlik eşiğin altına düşünce birleştirme durur.
- Küme benzerliği ölçütleri: single link, complete link, group average link. Ayrıntılar veri madenciliği dersindedir.
- Yukarıdan aşağı (top-down) ya da aşağıdan yukarı (bottom-up) kümeleme yapılabilir.

Boyutun laneti

- Çok boyutlu vektörlerle işlem süresi çok artar.
- Slayttaki kutu: boyut 1 iken verinin %50'si merkeze yakın bir yerde toplanır. Boyut arttıkça veriler üstel olarak birbirinden uzaklaşır. İki durumda da sınıflandırma ve kümeleme zorlaşır.
- Teorik örnek sayısı: 1 boyut için 10, 2 boyut için 100, 3 boyut için 1000, 4 boyut için 10.000 örnek. Üstel artar. Pratikte buna ulaşılamaz ama bir oran gözetilmelidir.
  - Örnek sayısı azsa boyut sayısı azaltılmalıdır.
  - Boyut azken örnek çok fazlaysa örnekler rastgele seçilerek azaltılabilir.
- Verilmesi gereken kararlar:
  - Kök mü, gövde mi, kelimenin tamamı mı kullanılacak?
  - Stop words ve function words atılacak mı?
  - Filtre ya da sarmalayıcı yöntemle özellik seçilecek mi?
  - Uzayı değiştiren yöntemler (LSI, PCA, LDA) kullanılacak mı?
- Seyrek kelimeler: tür belirlemede bir dokümanda tek kez geçen kelimenin pek yararı yoktur, atılabilir. Yazar tanımada ise yalnızca bir yazarın kullandığı düşük frekanslı kelimeler çok ayırt edicidir.
- Kelime frekans dağılımı grafiği:
  - Çok yüksek frekanslı kelimeler önemsizdir (stop words).
  - Çok düşük frekanslı kelimeler kişiye özel kullanımlar olabilir.
  - Ortadaki kelimeler önemlidir.
  - Hangi bölgenin önemli olduğu uygulamaya göre değişir.

Kelime torbası (bag of words)

- Derlemdeki bütün tekil kelimelerden oluşan yapıya (sözlüğe) kelime torbası denir. Dokümanda geçen bütün kelimelerin bulunduğu bir çanta gibi düşünülür.

Abstract Feature Extraction (AFE)

- Hikâyesi:
  - Bir lisans dersinde verilen ödevde bir öğrenci ilk fikri geliştirdi. Ulusal bir yayın yapıldı, sonra unutuldu.
  - Yıllar sonra Göksel Biricik doktorasında takıldığında hoca bu fikri hatırladı. Üzerinde birlikte çalıştılar. Yöntemin ilk adı "Yıldız"dı.
  - Dergiler sonuçların bu kadar iyi olmasına inanmadı: "Kendi veri setinizde iyi çalışıyor" dediler.
  - Hakemlerin isteğiyle sırasıyla Reuters (o zaman postayla CD olarak gelmiş), ModApte ve 20 Newsgroups veri setlerinde denendi. Her seferinde iyi sonuç alındı ve sonunda kabul edildi.
  - O dönem derin öğrenme yoktu. Makine öğrenmesi yeni yeni gündeme geliyordu.
- Özellikleri:
  - Öz vektör ve öz değer kullanılmaz.
  - Tekil değer ayrıştırması (Singular Value Decomposition) yapılmaz.
  - Yalnızca terimlerin ağırlıkları ve sınıflar üzerindeki olasılık dağılımları kullanılır.
  - Terim olasılıklarının sınıflara izdüşümü alınıp toplanarak her terimin sınıfları ne kadar etkilediği bulunur.
- Gösterim:
  - i terim, j doküman, k sınıf indisi.
  - $n_{i,j}$: t_i teriminin d_j dokümanında kaç kez geçtiği.
  - N_i: t_i terimine sahip doküman sayısı. N: derlemdeki toplam doküman sayısı.
- Formüller:
  - t_i teriminin c_k sınıfında kaç kez geçtiği: $nc_{i,k} = \sum_j n_{i,j}, \quad d_j \in c_k$
  - t_i teriminin c_k sınıfını ne kadar etkilediği: $w_{i,k} = \log(nc_{i,k} + 1) \times \log\left(\frac{N}{N_i}\right)$
  - "+1" sıfır sayımları için add-one yumuşatmasıdır. $\log(N/N_i)$ normalizasyon katsayısıdır. Hoca, bu veride en iyi sonucu bu katsayının verdiğini söyledi. Başka katsayılar da denenebilir.
  - Bir dokümandaki bütün terimlerin c_k sınıfına etkisi: $Y_{j,k} = \sum_{t_i \in d_j} w_{i,k}$
  - Normalize edilmiş yeni özellik: $yeniTerim_{j,k} = \frac{Y_{j,k}}{\sum_k Y_{j,k}}$

AFE örneği

- İki sınıfta toplam 6 doküman ve 8 terim var.
  - Sınıf 1: B1 (t1, t2, t3), B2 (t2, t4, t8), B3 (t2, t2, t5).
  - Sınıf 2: B4 (t5, t6), B5 (t5, t7, t8), B6 (t2, t5, t6).
- Terim-doküman matrisi:

Terim	d1	d2	d3	d4	d5	d6
Terim1	1	0	0	0	0	0
Terim2	1	1	2	0	0	1
Terim3	1	0	0	0	0	0
Terim4	0	1	0	0	0	0
Terim5	0	0	1	1	1	1
Terim6	0	0	0	1	0	1
Terim7	0	0	0	0	1	0
Terim8	0	1	0	0	1	0

- Terim 5 için (logaritmalar 10 tabanında):
  - $N = 6$, $N_5 = 4$ (d3, d4, d5, d6).
  - Sınıf 1'de (d1, d2, d3) 1 kez, sınıf 2'de (d4, d5, d6) 3 kez geçiyor.
  - $w_{5,1} = \log(1+1) \times \log(6/4) \approx 0{,}053$
  - $w_{5,2} = \log(3+1) \times \log(6/4) \approx 0{,}106$
- Terim 6 için: $N_6 = 2$. Sınıf 1'de 0, sınıf 2'de 2 kez geçiyor.
  - $w_{6,1} = \log(1) \times \log(3) = 0$

  - $w_{6,2} = \log(3) \times \log(3) \approx 0{,}228$
- Doküman 4 (t5 ve t6 içeriyor):
  - $Y_1 = 0{,}053 + 0 = 0{,}053$
  - $Y_2 = 0{,}106 + 0{,}228 = 0{,}334$
  - $yeniTerim_1 = 0{,}053/(0{,}053 + 0 + 0{,}106 + 0{,}228) = 0{,}137$
  - $yeniTerim_2 = 0{,}334/0{,}387 = 0{,}863$
  - Yorum: doküman 4 yaklaşık %14 sınıf 1'e, %86 sınıf 2'ye aittir.
- Bütün dokümanlar için sonuç:

Doküman	Özellik 1 (Sınıf 1)	Özellik 2 (Sınıf 2)
Doküman 1	0,918	0,082
Doküman 2	0,718	0,282
Doküman 3	0,585	0,415
Doküman 4	0,137	0,863
Doküman 5	0,289	0,711
Doküman 6	0,313	0,687

- Doküman 1-3 sınıf 1'de, 4-6 sınıf 2'de daha yüksek değer alıyor. Doğru ayrılıyorlar.
- AFE aynı zamanda bir özellik çıkarımıdır. Özellik sayısını sınıf sayısına indirir (burada 8 terimden 2 özelliğe).
- Avantajları: etiketli veriyle kullanılır, arkasında ağır bir matematik yoktur, boyutu kesinlikle sınıf sayısına indirir.
- Dezavantajı: gelen dokümanın hiçbir terimi eğitim verisinde yoksa her sınıfa %0 ağırlık çıkar. Doküman sınıflandırılamaz. Bu durumda doküman derleme katılıp sistem yeniden eğitilmelidir. Dile yeni terimler de girdiği için sistem aralıklarla yeniden eğitilmelidir.

Kelime koordinatları yöntemi

- Elde embedding yoksa iki kelimenin benzerliği şöyle bulunabilir:
  - İki kelime arama motoruna verilir.
  - Arka arkaya, aynı cümlede ya da aynı dokümanda kaç kez birlikte geçtiğine bakılır (tercihe bağlı).
- Örnek birlikte geçme sayıları (Google):

	akbaba	ayı	baykuş	araba	limuzin
akbaba		0	20	1	0
ayı	0		33	4	0
baykuş	20	33		0	0
araba	1	4	0		38
limuzin	0	0	0	38	

- Bu matris çok boyutlu ölçekleme (Multidimensional Scaling, MDS) ile iki boyutlu düzleme yerleştirilir (MATLAB'de kütüphanesi var).
- Sonuç: limuzin ve araba bir tarafta (taşıt), ayı, baykuş ve akbaba diğer tarafta (hayvan) toplanır. Yalnızca birlikte geçme sayılarıyla ayrım yapılabilir.
- Deneyler genişletilmiş:
  - Veri kümesi 1: yer, hayvan, taşıt (Google). Sınıflar ayrılıyor. Örneğin "at" kelimesi taşıt kümesine yakın çıkıyor, çünkü birlikte kullanılıyorlar.
  - Veri kümesi 2: ev eşyası, hayvan, giyecek. "Merdiven" ve "divan" ortada bir yere kaymış.
  - Veri kümesi 3: 6 sınıf (yiyecek, giyecek, ev eşyası, hayvan, yer, taşıt), hem Yahoo hem Google ile. Google ile sonuçlar daha iyi, çünkü indeksleme mekanizması daha iyiydi.
- İyileştirme: "akbaba" ile "ayı" ikisi de hayvan olduğu hâlde birlikte hiç geçmemiş (0). Ham sayım yerine normalize edilmiş değer kullanılır:

$$\frac{n(A,B)}{n(A) \times n(B)}$$

- Örnek: A 3 kez, B 5 kez, ikisi birlikte 7 kez geçmişse değer $\frac{7}{3\times5}$ olur.
- Farklı sınıflandırma yöntemleri ve boyutlarla sonuçlar çok değişiyor. Örneğin iki boyutta Expectation Maximization çok daha iyi sonuç vermiş. Bu yine çapraz geçerlemenin ve yöntem denemenin önemini gösterir.

TF-IDF ağırlıklandırma

- IDF: Inverse Document Frequency (ters doküman frekansı).
- Örnek: 4 doküman var.
  - A kelimesi dokümanlarda 7, 3, 5 ve 6 kez geçiyor. Toplam 21. Dört dokümanın hepsinde var.
  - B kelimesi 1, 0, 0 ve 10 kez geçiyor. Toplam 11. İki dokümanda var.
- Yalnızca terim frekansına bakılırsa A (21) daha önemli görünür. Bu yanlıştır.
- TF-IDF: $w = tf \times \log \frac{N}{n}$ (N: toplam doküman sayısı, n: kelimenin geçtiği doküman sayısı). Hocanın hesabında logaritma 2 tabanındadır.
  - A: $21 \times \log(4/4) = 21 \times 0 = 0$. Her dokümanda geçtiği için ayırt edici değildir.
  - B: $11 \times \log_2(4/2) = 11 \times 1 = 11$. B daha ayırt edicidir.
- Bazı uygulamalarda ham terim frekansı yerine TF-IDF kullanmak çok önemlidir. Hangisinin daha iyi olduğu denenerek bulunur.

İzdüşüm (projeksiyon) yöntemleri

- AFE, PCA, LDA ve LSI (Latent Semantic Indexing) boyutu değiştiren, yani izdüşüm alan yöntemlerdir.
- Örnek: kırmızı ve mavi noktalar bir eksene izdüşürülünce karışır. Başka bir eksene izdüşürülünce kırmızılar bir tarafta, maviler diğer tarafta toplanır ve kolayca ayrılır.
- LSI bilgi erişimi (information retrieval) konusunda ayrıca anlatılacak.
- Kelimeler frekanslarına göre renklendirilerek görselleştirilebilir. Mavi kelimeler az, koyu renkli uçtakiler çok sık geçen kelimelerdir.

Naive Bayes ile metin sınıflandırma

- Birçok öğrencinin projesinde kullanacağı yöntemdir.
- Gelen dokümanın her sınıfı hangi olasılıkla temsil ettiği bulunur. En yüksek değeri veren sınıf seçilir:

$$v_{NB} = \arg\max_{v_j \in V} P(v_j) \prod_i P(w_i|v_j)$$

- $P(v_j)$: sınıfın olasılığı. Çarpım: sınıftaki terimlerin olasılıklarının çarpımı.
- Terim frekansı sıfır olabileceği için yumuşatma uygulanır:

$$P(w_k|v_j) = \frac{n_k + 1}{n_j + |V|}$$

- n_k: w_k kelimesinin ilgili sınıftaki sayısı.
- n_j: j. sınıftaki kelime sayısı.
- $|V|$: sözlükteki kelime sayısı.

Naive Bayes örneği

- Eğitim dokümanları:
  - Sınıf A: “cat”, “crab”, “crawl” ve “stairs” kelimelerini içeren bir cümle (altyazıda tam cümle anlaşılmıyor). Stop words ve ekler atılınca kalan kelimeler: cat, crab, crawl, stair (4 kelime).
  - Sınıf B: “It’s raining cats and dogs”. Kalan kelimeler: rain, cat, dog (3 kelime).
- Ortak sözlük: cat, dog, crab, crawl, stair, rain. $|V| = 6$.
- Sınıf olasılıkları: $P(A) = 1/2$, $P(B) = 1/2$.
- Test dokümanında sözlükte bulunan kelimeler: cat ve dog. Diğer kelimeler sözlükte yok.

- A sınıfı için:
  - $P(\text{cat}|A) = \frac{1+1}{4+6} = \frac{2}{10}$
  - $P(\text{dog}|A) = \frac{0+1}{4+6} = \frac{1}{10}$
  - $P(A) \times P(\text{cat}|A) \times P(\text{dog}|A) = \frac{1}{2} \times \frac{2}{10} \times \frac{1}{10} = 0{,}01$
- B sınıfı için:
  - $P(\text{cat}|B) = \frac{1+1}{3+6} = \frac{2}{9}$
  - $P(\text{dog}|B) = \frac{1+1}{3+6} = \frac{2}{9}$
  - $P(B) \times P(\text{cat}|B) \times P(\text{dog}|B) = \frac{1}{2} \times \frac{2}{9} \times \frac{2}{9} \approx 0{,}0247$
- $0{,}0247 > 0{,}01$ olduğu için test dokümanı B sınıfına aittir.
- Bu, Naive Bayes'in en basit hâlidir. Multinomial Naive Bayes özellik değerlerini algoritma içinde değiştirir ve bazen daha iyi sonuç verir.

Metin sınıflandırmanın özeti

- Doğru özellikler seçilmeli.
- Özellik sayısı ile örnek sayısı arasındaki oran ayarlanmalı.
- Gereksiz özellikler özellik seçimiyle atılmalı.
- Veriler çok örtüşüyorsa izdüşüm yöntemleriyle başka bir uzaya taşınmalı.
- Her yöntem ailesinden birer yöntem denenip en uygun model bulunmalı.
- Eğitim ve testte çapraz geçerleme yapılmalı.
- Bunlar düzgün yapılırsa kötü sonuç almak zordur. Ama veri setinin içeriği de çok önemlidir.

Ders sonrası notlar

- Skip-gram slaytında pencere boyutu 7 (sağdan 3, soldan 3) yazıyor. Hoca tahtada çizerken her iki yandan 2 kelime alıp pencereyi 5 olarak gösterdi. İkisi de geçerli bir seçimdir, örnekler arasında fark buradan kaynaklanıyor.
- Hoca CBOW'un küçük, skip-gram'ın büyük veri setlerinde daha iyi olduğunu söyledi. word2vec'in özgün açıklamasında (Mikolov) bunun tersi geçer: skip-gram az veriyle iyi çalışır ve nadir kelimeleri iyi temsil eder; CBOW çok daha hızlı eğitilir ve sık kelimelerde biraz daha iyidir.
- Hoca BERT derlemi için "8 milyon kelimelik" bir derlemden bahsetti. BERT, yaklaşık 800 milyon kelimelik BooksCorpus ile 2,5 milyar kelimelik İngilizce Wikipedia üzerinde eğitilmiştir.
- Hoca, Google'ın BERT-Base modelini kapattığını ve ücretli verdiğini duyduğunu söyledi. Google'ın yayımladığı BERT modelleri açık kaynak lisansla dağıtılmaya devam ediyor. Ücretli olan, modeli eğitmek ya da çalıştırmak için kullanılan bulut hizmetleridir (TPU vb.).
- Hoca bilgi kazancı için "entropi ne kadar büyükse o kadar iyi bir özelliktir" dedi. Doğrusu: bilgi kazancı, bir özelliğin bilinmesiyle sınıf entropisinde sağlanan azalmadır. Bilgi kazancı ne kadar büyükse, yani özellik bilindiğinde kalan entropi ne kadar küçükse, özellik o kadar iyidir.
- Hoca bir yerde "LSI, PCA sınıf bilgisi olan yöntemler" dedi, hemen ardından tersini söyledi. Doğrusu: PCA ve LSI sınıf bilgisi kullanmaz (eğiticisiz). LDA (Linear Discriminant Analysis) sınıf bilgisi kullanır. Hocanın daha önceki "LDA etiketli bir yöntem" ifadesi doğrudur.
- Hoca uzaklık için "bir bölü benzerlik" dedi. $[0, 1]$ aralığında çalışılıyorsa genelde uzaklık $d = 1 - s$ olarak alınır. $1/s$ kullanılsaydı benzerlik 0 iken uzaklık sonsuz olurdu.
- Hoca YATSI kısaltmasının açılımını hatırlayamadı. Açılımı "Yet Another Two-Stage Idea"dır.
- Hoca frekans dağılımı grafiğini anlatırken "sol taraf önemsiz ve düşük frekanslı, stop words" dedi. Stop words yüksek frekanslı kelimelerdir. Düşük frekanslı uç ise kişiye özel ya da nadir kelimelerdir.
- Altyazıda geçen bazı terimlerin doğruları: "SIBO/Sibov" → CBOW, "hat/hatvan gösterimi" → one-hot, "Mikalov" → Mikolov, "Glow/Globe" → GloVe, "Ludus" → Loodos, "Rapper/röper" → wrapper, "Vega" → Weka, "K-MINS/Kamins" → k-means, "Rolter/Reuter" → Reuters, "ModUp" → ModApte, "Nayibayes" → Naive Bayes, "Advanced smoothing" → add-one smoothing.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.